

PROJECT SYNOPSIS

Title of the Project

ReLoop: A Peer-to-Peer Web Marketplace for Second-Hand Items with Secure Escrow Payments

Submitted by	Tajibul Islam
Roll Number	145
Programme	B.Tech in Computer Science & Engineering
Semester / Section	4th Semester, Section B
Institution	Assam downtown University, Guwahati

1. Statement of the Problem

Across Indian cities, including Guwahati, a large volume of usable furniture, electronics, appliances, books, and clothing is discarded or left idle simply because sellers and buyers have no convenient, trustworthy channel to find one another. Existing options are fragmented: informal WhatsApp and Facebook groups have no search, no price protection, and no accountability if a buyer never receives what was promised or a seller never receives payment; classifieds sites list an item but leave price negotiation, meeting arrangements, and payment entirely outside the platform, offering no recourse when something goes wrong.

This creates two linked problems. First, genuinely usable goods are under-utilised or thrown away, adding avoidable pressure on landfills and manufacturing demand, when reuse would have served the same need at lower cost and lower environmental impact. Second, the buyers and sellers who do attempt second-hand transactions online take on real financial risk, since no independent party holds the payment until both sides confirm the exchange went as agreed. There is a clear need for a single web platform that lets people list, discover, negotiate, and safely pay for second-hand items in one place, with a neutral escrow step and a mediator to fall back on when a transaction goes wrong.

2. Objective and Scope of the Project

The objective of this project is to design and develop ReLoop, a fully functional web-based marketplace exclusively for second-hand goods, that takes a buyer from search to a safely completed purchase without leaving the platform. The specific objectives are:

- Allow any registered user to list a used item with photographs, a category, a standardised condition grade, a price, and a pickup/delivery location.
- Let buyers search and filter listings by keyword, category, price range, and location, and save searches to be alerted when a new matching item is listed.
- Support direct price negotiation through an offer / counter-offer / accept-or-reject workflow between buyer and seller.
- Provide in-app, per-listing messaging so buyers and sellers can coordinate without exchanging personal contact details up front.

- Process payment through a card-style checkout that is held in escrow and released to the seller only once the buyer confirms receipt, with an admin-mediated dispute path for refunds when something goes wrong.
- Give administrators an operational dashboard: open disputes to resolve, and platform-wide analytics on listings, users, and revenue.

Scope: the project covers the full transaction lifecycle for second-hand goods — listing, discovery, negotiation, messaging, escrow payment, and dispute resolution — for a single-region, single-currency deployment. It does not cover logistics/courier integration, multi-seller storefronts, or real bank/card settlement; these are identified under Future Work.

3. Methodology / Process Description

The system follows a standard three-tier web architecture: a browser-based frontend, a REST API backend, and a relational database, communicating over HTTPS with session-based authentication.

Process flow of a typical transaction:

- 1. Seller registers/logs in and submits the listing form (title, description, category, condition, price, location, photo).
- 2. The backend validates and stores the listing; it becomes immediately searchable.
- 3. Buyer searches/filters listings, opens a listing, and either sends a price offer, messages the seller, or proceeds directly to checkout.
- 4. If negotiating: seller accepts, counters, or rejects the offer; buyer accepts or rejects any counter-offer.
- 5. On checkout, the backend simulates a card charge and creates a transaction record with status ESCROW; the listing is marked sold.
- 6. Buyer confirms receipt (or, for local pickup, confirms the in-person exchange) — the transaction status changes to RELEASED and payout to the seller is recorded as instant.
- 7. If the buyer or seller raises a problem instead, the transaction status changes to DISPUTED and an administrator resolves it — release to seller, refund buyer, or partial refund — through the admin dashboard.

This flow was implemented with the following module breakdown, each exposed as an independent REST route group: Authentication, Listings (CRUD + search), Offers (negotiation), Messages (chat), Wishlist (saved items/searches), Payments (escrow/checkout/disputes), and Admin (dispute queue + analytics). Passwords are never stored in plain text — they are hashed with bcrypt before being written to the database, and every state-changing route checks that the requesting user actually owns or is party to the resource being modified.

4. Hardware & Software Requirements

Component	Requirement
Development machine	Any laptop/desktop with 4 GB+ RAM, Windows/Linux/macOS
Runtime	Node.js (v18+) with the Express.js framework
Database	SQLite (via better-sqlite3) for development; portable to PostgreSQL/MySQL for production
Frontend	HTML5, CSS3, vanilla JavaScript (responsive, mobile-friendly layout)

Supporting libraries	bcrypt.js (password hashing), Multer (image uploads), express-session (auth sessions)
Payment gateway	Stripe Checkout + Stripe Connect (escrow/payouts) in production; simulated in the current build for the reasons noted in Section 6
Version control / hosting	Git/GitHub for source control; any Node-compatible host (Render, Railway, or a VPS) for deployment

5. Project Schedule

The project was executed over an 8-week schedule, structured as follows:

Week	Phase	Deliverable	Status
1	Requirement analysis & synopsis	Approved project synopsis	Complete
2	Database & API design	Schema (8 tables) + route plan	Complete
3	Auth & listings module	Register/login, create/search listings	Complete
4	Negotiation & messaging	Offers, counter-offers, in-app chat	Complete
5	Payments & escrow	Checkout, escrow hold, confirm-receipt payout	Complete
6	Wishlist, alerts & sharing	Saved searches, wishlist, social share	Complete
7	Admin dashboard & disputes	Dispute queue, resolution, analytics	Complete
8	Testing, styling & deployment prep	End-to-end test pass, responsive UI, README	Complete

6. Future Work

- Replace the simulated payment step with a live Stripe Checkout + Stripe Connect integration for real card settlement and automated seller payouts.
- Add courier/logistics-partner integration so 'delivery' orders can be tracked, not just marked as shipped.
- Government-ID verification service integration for the Verified Seller badge, instead of the current self-service demo flag.
- Real-time messaging via WebSockets, and push/email notifications for offers, messages, and saved-search alerts.
- A recommendation engine that surfaces listings based on browsing and purchase history.
- Native mobile apps (Android/iOS) built on the same REST API.

7. Resources and Limitations

Resources used: a standard development laptop, Node.js runtime, open-source npm packages (Express, better-sqlite3, Multer, bcrypt.js), and free-tier hosting for demonstration. No proprietary or licensed software was required.

Limitations: the current build uses SQLite, which is appropriate for demonstration and moderate load but would need migration to PostgreSQL/MySQL for large-scale concurrent production use. The payment gateway is simulated rather than live, as noted above. Seller ID verification is a self-service flag rather than a checked government-ID upload. The system is built for a single region/currency and does not yet integrate a courier/logistics partner for delivery tracking.

8. Conclusion

ReLoop demonstrates that a complete, trustworthy second-hand marketplace — covering listing, search, negotiation, messaging, escrow payment, and dispute resolution — can be built end-to-end with a compact, well-structured full-stack codebase. What distinguishes it from a simple listings site is the escrow-and-dispute layer: buyers are protected because funds are held until they confirm receipt, and sellers are protected because an admin-mediated process exists if a claim is disputed, rather than either party being left to resolve disagreements informally. Combined with practical touches such as saved-search alerts, standardised condition grading, and a visible carbon-footprint estimate per listing, the project delivers a marketplace that is both functionally complete and oriented toward encouraging genuine reuse of goods.

Questions for Submission

Q1. Why should the department approve this project?

This project directly applies core Full Stack Web Development internship concepts — REST API design, relational database modelling, session-based authentication, and responsive frontend development — to a problem with genuine real-world relevance: enabling safe, trustworthy re-use of second-hand goods. Unlike a purely academic exercise, it required solving problems that any production e-commerce system must solve, such as protecting both sides of a transaction with an escrow mechanism and providing a fair dispute-resolution path, which pushed the work well past a basic CRUD application. It also demonstrates the ability to design a multi-module system (eight interlocking database tables and seven API route groups) and deliver it as a working, testable, end-to-end product rather than a set of disconnected screens, which is the kind of practical, deployable output the department's internship track is meant to produce.

Q2. What are the strengths and limitations of the proposed work?

Strengths: the system is functionally complete across the entire transaction lifecycle (list → search → negotiate → message → pay → resolve disputes); passwords are properly hashed rather than stored in plain text; every state-changing action is checked against the requesting user's ownership/participation before it is allowed; and the codebase is modular, so each concern (auth, listings, offers, messaging, payments, admin) can be tested, extended, or replaced independently. Limitations: the payment gateway is simulated rather than connected to a live processor, seller identity verification is a self-declared flag rather than a checked document upload, the database (SQLite) is suited to demonstration-scale rather than high-concurrency production load, and there is no courier/logistics tracking for shipped items — all of which are scoped as concrete future work rather than open problems.

Q3. What will be the positive impact of this project on society/academics/college/industry?

On society, a working second-hand marketplace with genuine buyer protection encourages people to sell and buy used goods with confidence instead of discarding them, directly supporting waste reduction and the reuse economy at a local level, which matters in a growing city like Guwahati. Academically, the project gives a concrete, gradable demonstration of full-stack competency spanning database design, API architecture, authentication, and UI development, useful both as a portfolio piece and as a template other students could extend for related marketplace-style projects. For the college, it is a project that can be showcased to industry partners as evidence of practical, deployable skill rather than textbook exercises. For industry, the escrow-and-dispute pattern implemented here reflects a real and current gap in many regional classifieds platforms, and the project could plausibly be extended into an actual community or campus resale service.

References / Bibliography

- [1] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.
- [2] Express.js Documentation, "Express - Node.js web application framework," 2024. [Online]. Available: <https://expressjs.com/>
- [3] Stripe, Inc., "Stripe Connect Documentation," 2024. [Online]. Available: <https://stripe.com/docs/connect>
- [4] SQLite Documentation, "SQLite: Small, Fast, Self-Contained SQL Database Engine," 2024. [Online]. Available: <https://www.sqlite.org/docs.html>
- [5] Mozilla Developer Network, "Web APIs and Guides for HTML, CSS and JavaScript," 2024. [Online]. Available: <https://developer.mozilla.org/>
- [6] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2023. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [7] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2019.